Orca

Danyang Zhuo
Duke University

Some materials are from Orca's OSDI22 Slides and Haichen Shen's slides

Efficient inference

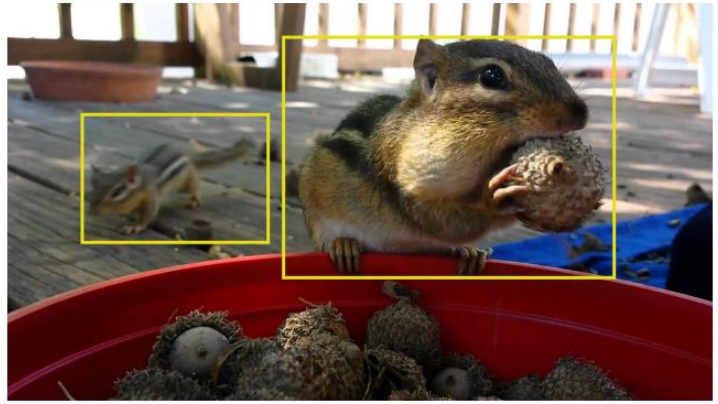
- Let’s first understand how inference works before the era of LLMs
- We then discuss how LLM inference works
- The goal is the same:
  - Minimize resource usage: # GPUs
  - Maximize performance: high throughput, low latency

Analyze video at large scale

Real-time traffic monitoring

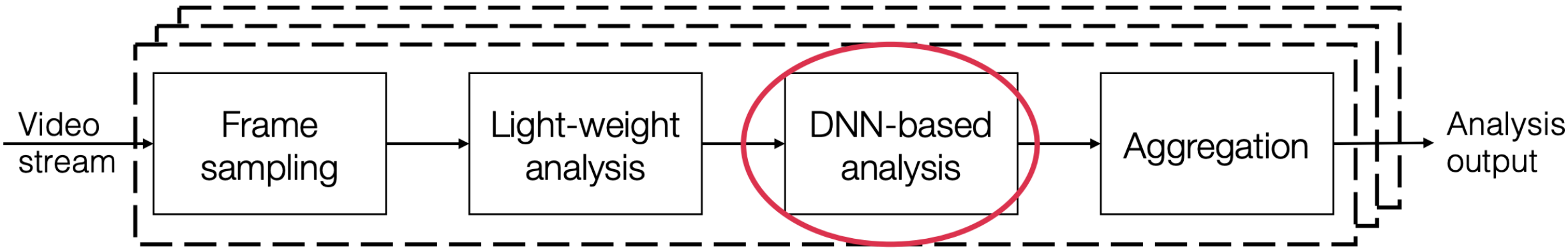
Game stream indexing

Surveillance



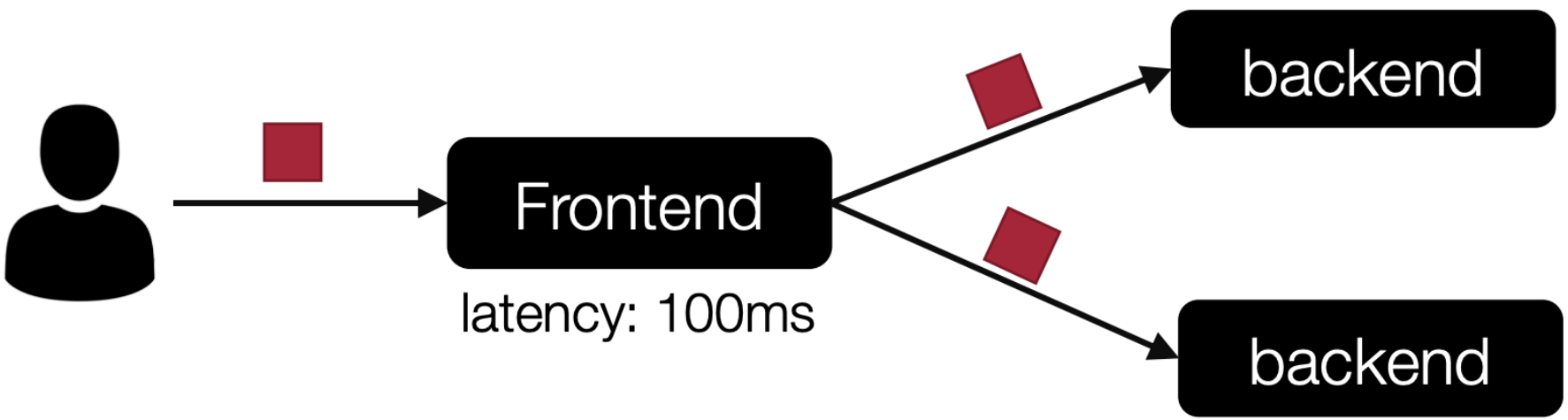
Intelligent family camera

Video analysis pipeline



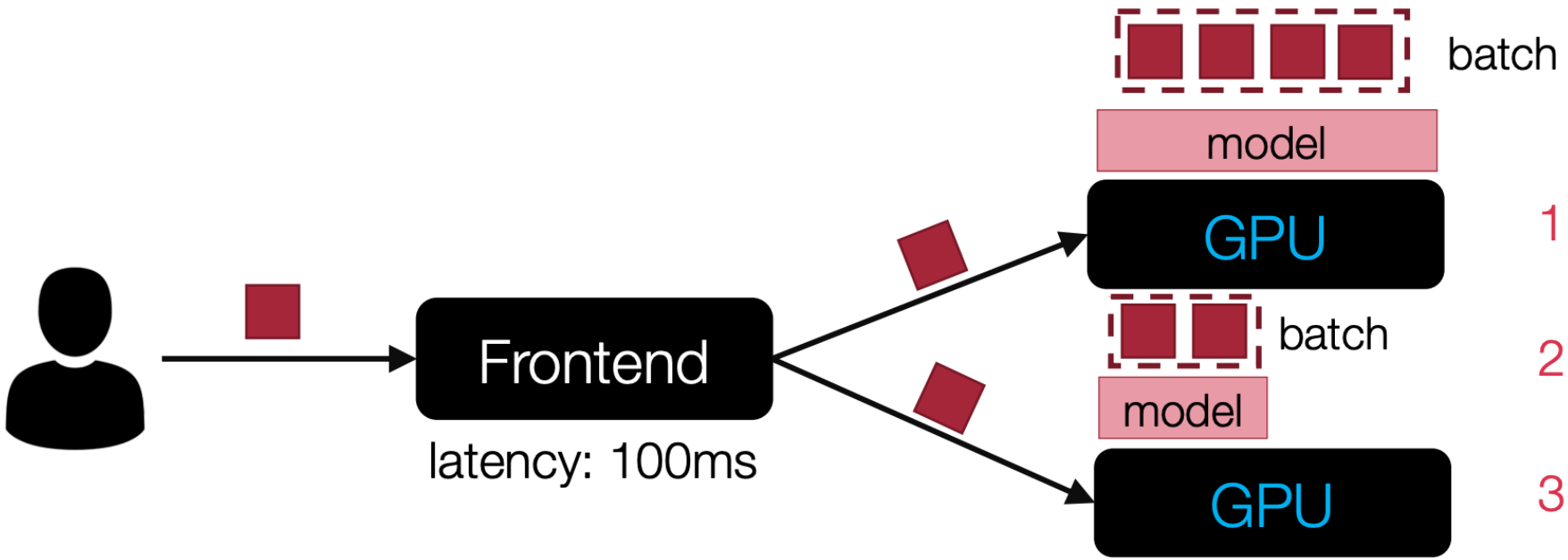
Most computation and cost

DNN serving similar to traditional distributed serving



- Auto scaling
- Load balancing
- Latency constraints

DNN serving imposes additional constraints



1. Use accelerators
2. Pre-load models
3. Batch processing

Nexus: A GPU Cluster Engine for Accelerating DNN-Based Video Analysis

Haichen Shen
Amazon Web Services

Lequn Chen
University of Washington

Yuchen Jin
University of Washington

Liangyu Zhao
University of Washington

Bingyu Kong
Shanghai Jiao Tong University

Matthai Philipose
Microsoft Research

Arvind Krishnamurthy
University of Washington

Ravi Sundaram
Northeastern University

Model	CPU lat. (ms)	GPU lat. (ms)	CPU cost ($) (0.1TF peak)	TPU cost ($) (180TF peak)	GPU cost ($) (125TF peak)
Lenet5	6	<0.1	$0.01	$0.00	$0.00
VGG7	44	<1	0.13	0.01	0.01
Resnet50	1130	6.2	4.22	0.48	0.12
Inception4	2110	7.0	8.09	0.93	0.23
Darknet53	7210	26.3	24.74	2.85	0.70

Table 1: DNN execution latencies and estimated costs per 1000 invocations.[1] Acceleration may be necessary to meet latency deadlines, but can also be cheaper, given low cost/TFLOPS.

Existing DNN serving systems are single-app solutions

E.g., Tensorflow Serving, Clipper

- Do not coordinate resource allocations across DNN applications
- Rely on external schedulers that cannot perform cross-app optimizations

How to build a serving system that coordinates the serving of multiple DNN applications?

Optimization opportunities

1. Cluster-level: batch-aware, latency-aware resource allocation across models

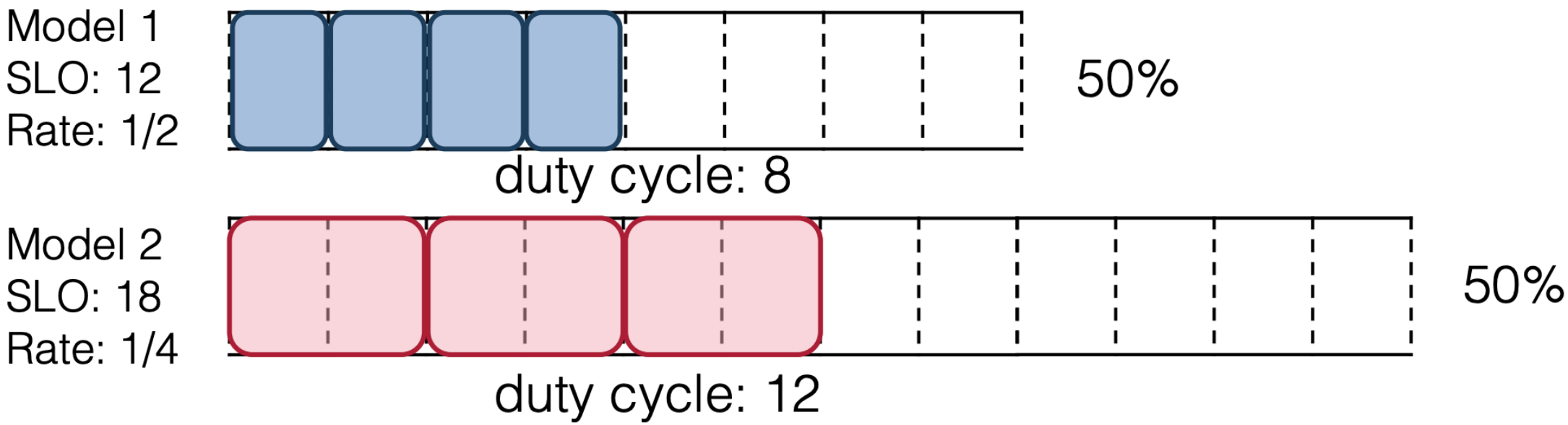
2. Application-level: handle complex queries

3. Model-level: batch at sub-model granularity

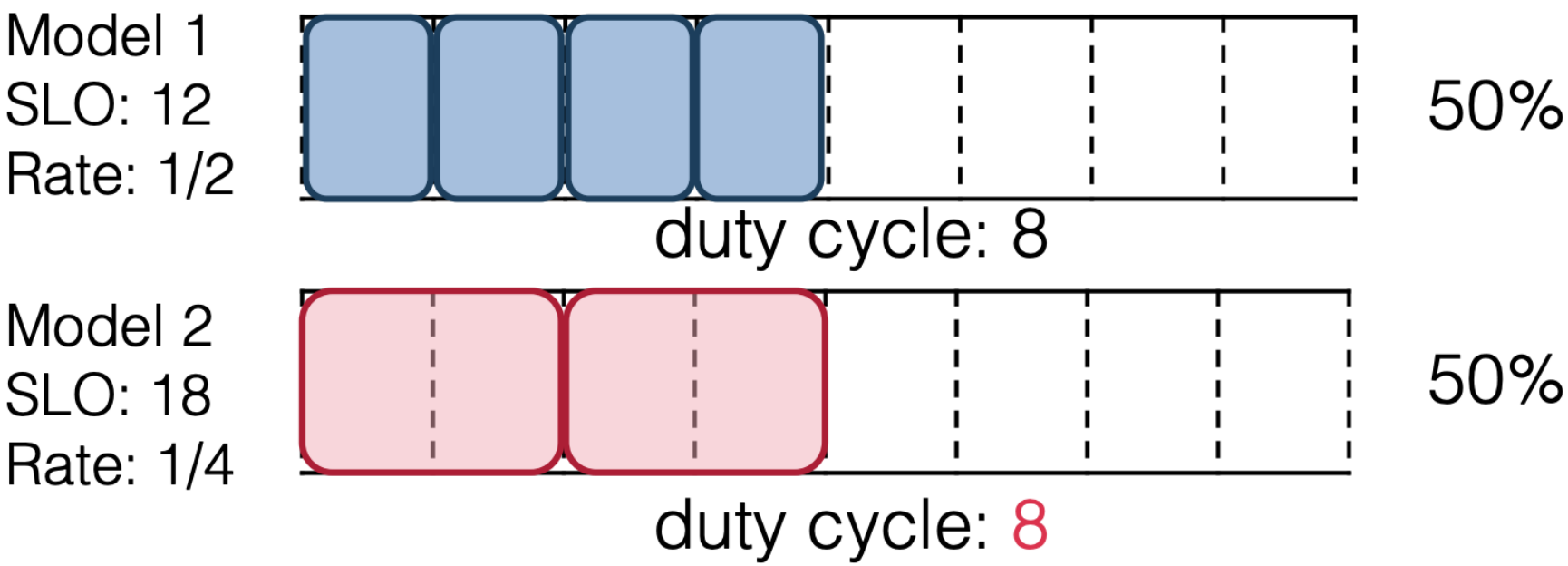
Model A			Model B			Model C		
Batch	Lat	Req/s	Batch	Lat	Req/s	Batch	Lat	Req/s
4	50	80	4	50	80	4	60	66.7
8	75	107	8	90	89	8	95	84
16	100	160	16	125	128	16	125	128

Table 2: Batching profiles for models used in the example. Lat is the latency (ms) for processing a batch, and Req/s is the throughput achieved.

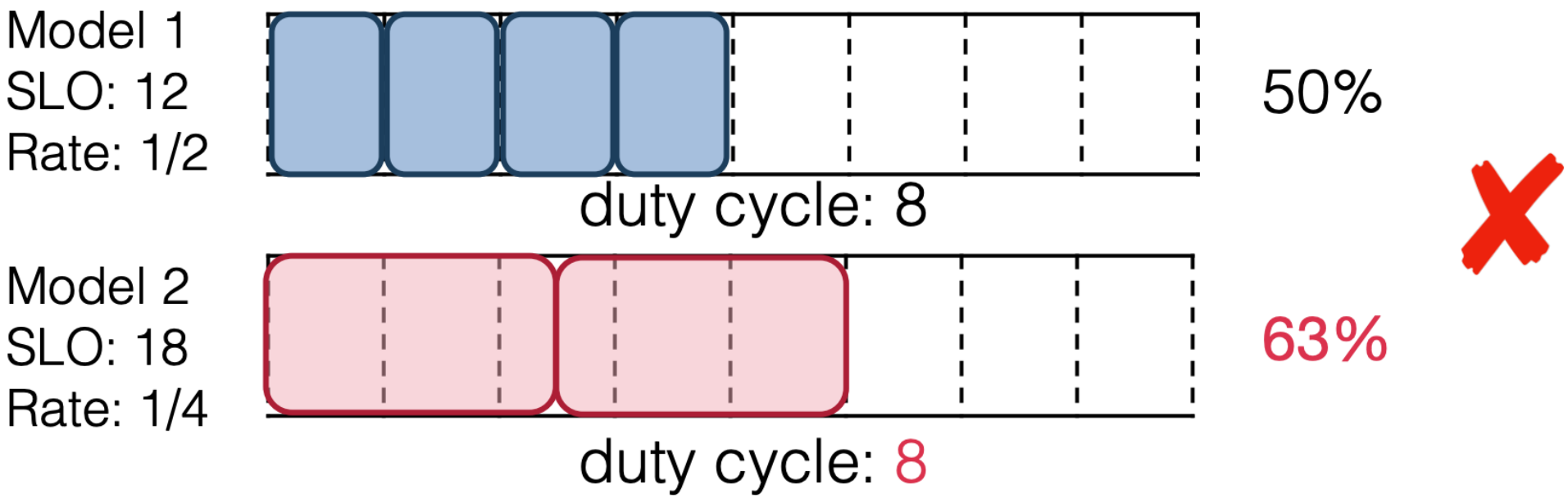
Opportunity 1: cluster-level resource allocation



Opportunity 1: cluster-level resource allocation

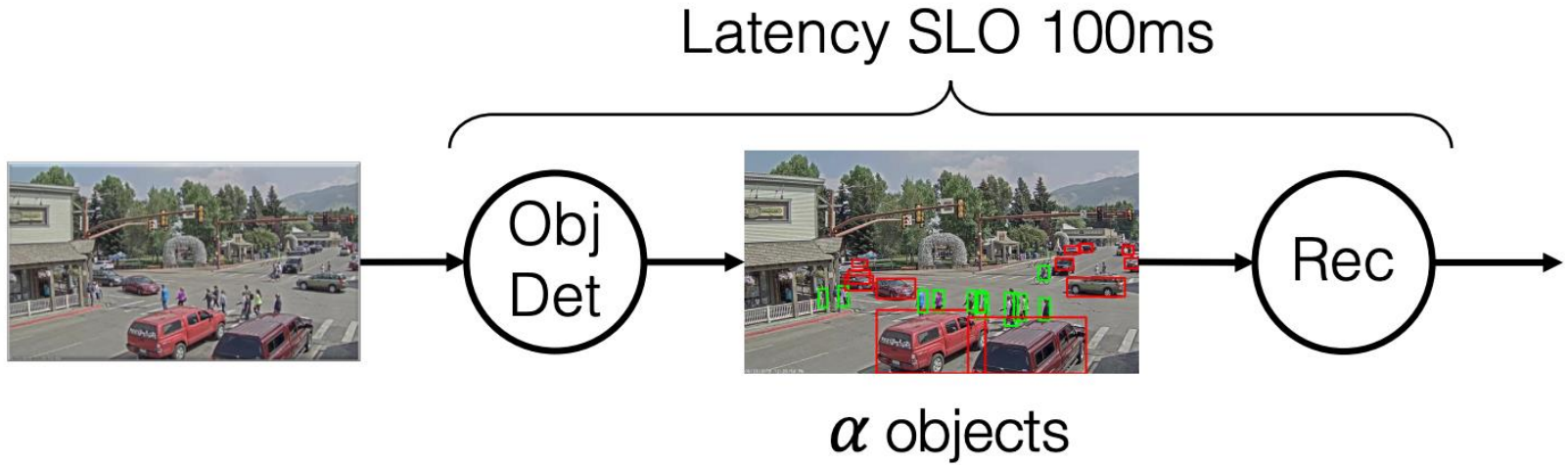


Opportunity 1: cluster-level resource allocation



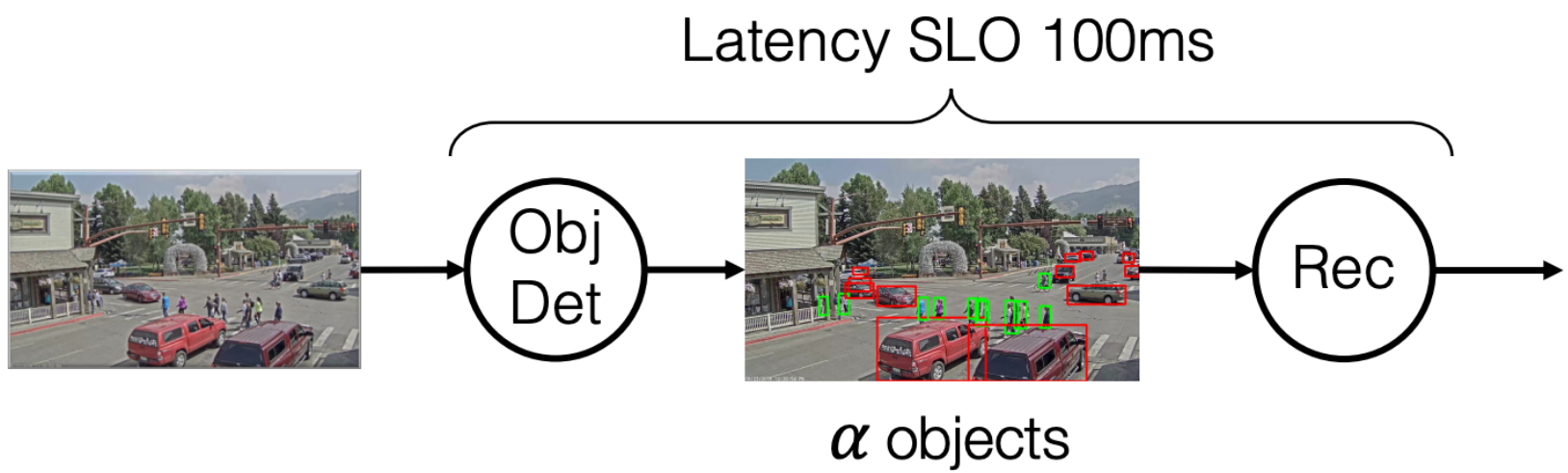
Challenge: GPU sharing has to account for SLO and "squishy" load demands across models

Opportunity 2: app-level complex query



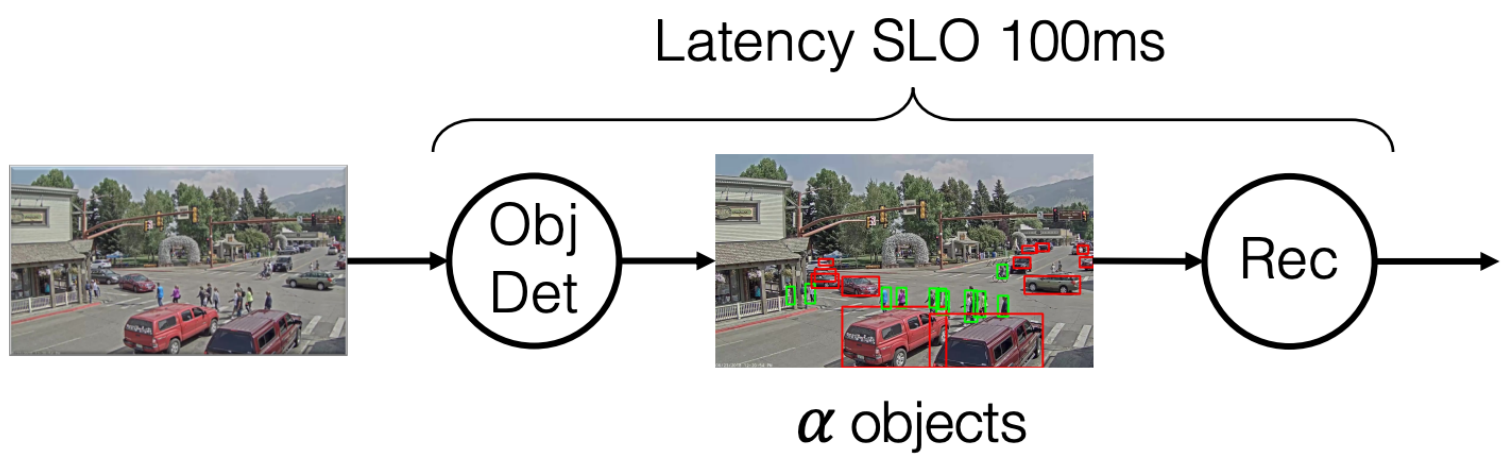
Model Latency SLO (ms)		Throughput (reqs/s/GPU)		
Detection	Recognition	$\alpha = 0.1$	$\alpha = 1$	$\alpha = 10$
40	60			
50	50			
60	40			

Opportunity 2: app-level complex query



Model Latency SLO (ms)		Throughput (reqs/s/GPU)		
Detection	Recognition	$\alpha = 0.1$	$\alpha = 1$	$\alpha = 10$
40	60	Low	Medium	High
50	50	Medium	High	Medium
60	40	High	Medium	Low

Opportunity 2: app-level complex query

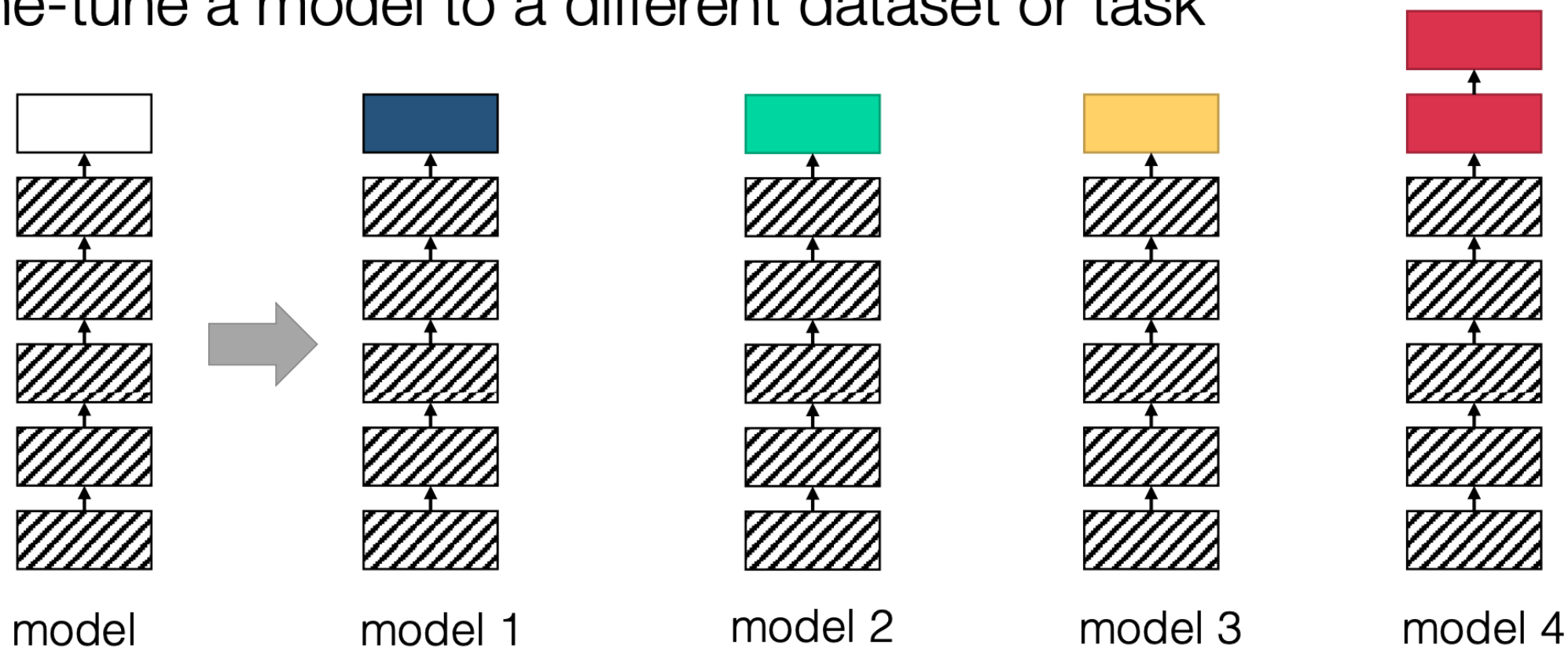


Model Latency SLO (ms)		Throughput (reqs/s/GPU)		
Detection	Recognition	$\alpha = 0.1$	$\alpha = 1$	$\alpha = 10$
40	60	Low	Medium	High

Challenge: Latency split impacts efficiency and needs to be adapted to workload

Opportunity 3: model-level transfer learning

- Fine-tune a model to a different dataset or task



Challenge: How to speed up the common part across models?

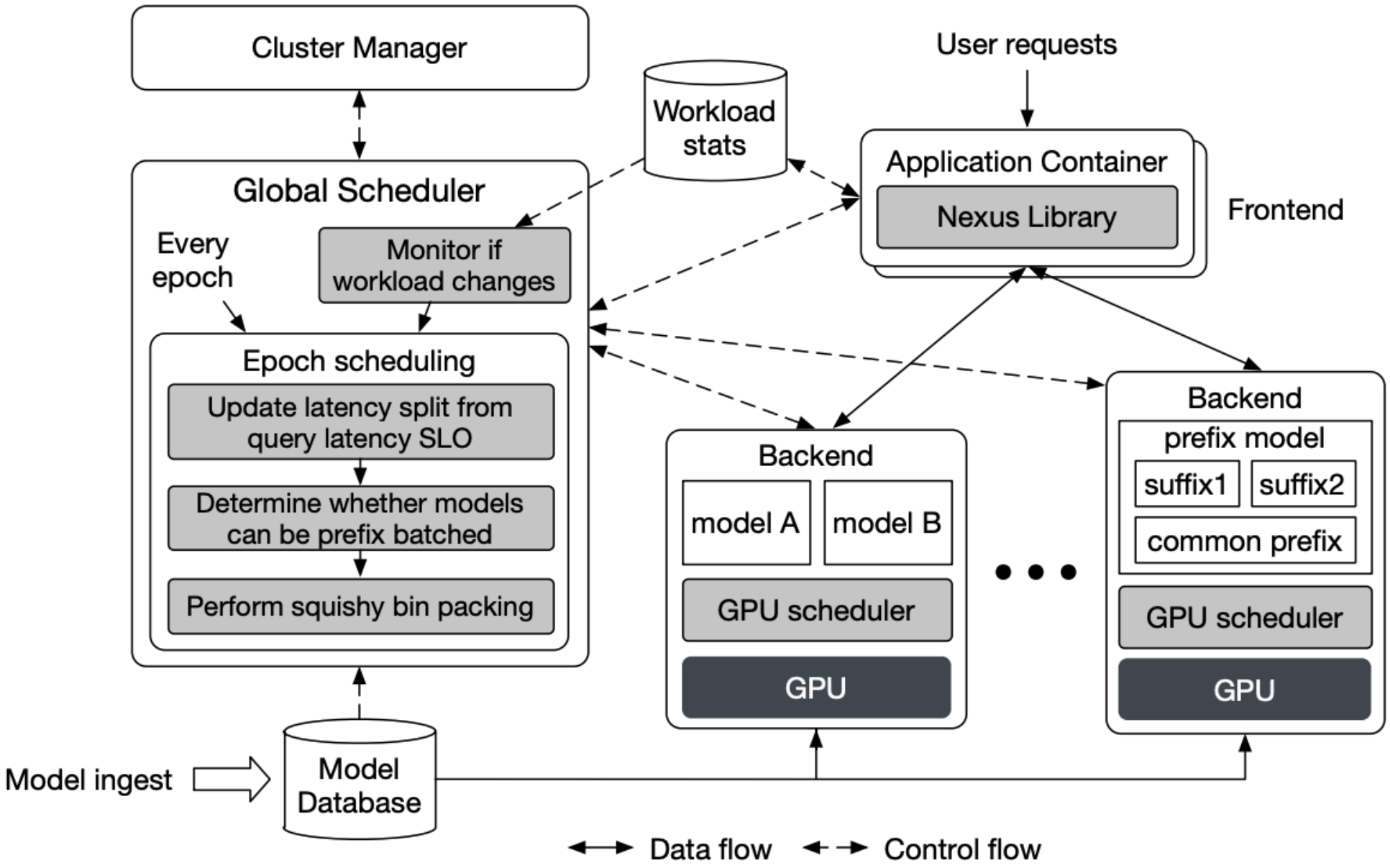


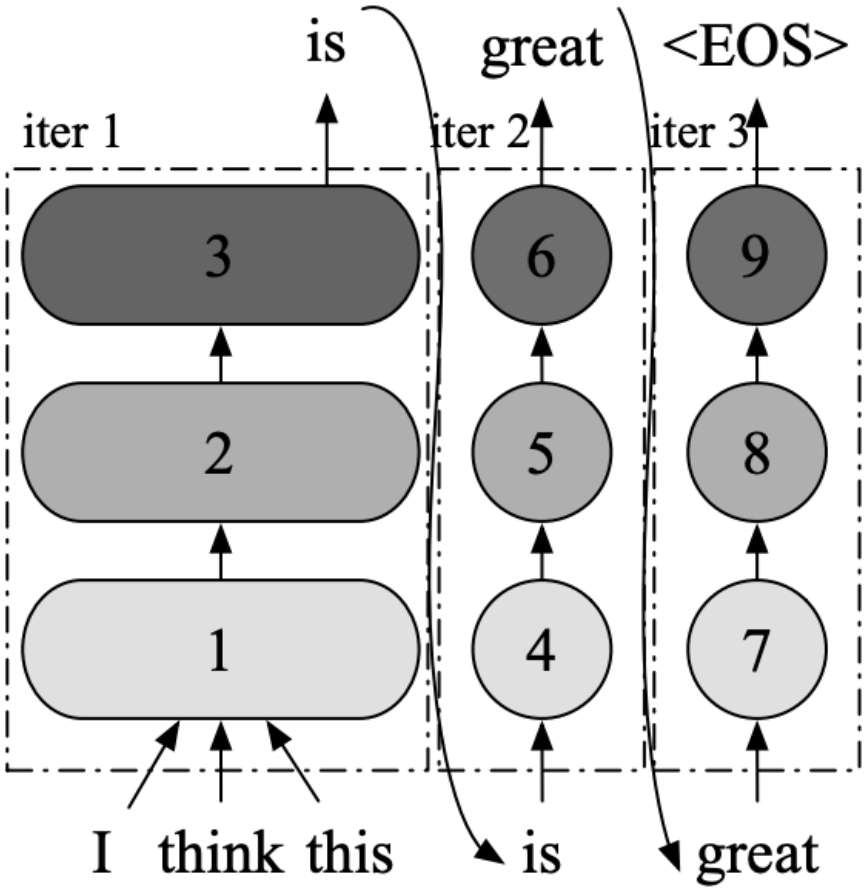
Figure 6: Nexus runtime system overview.

Resource allocation problem

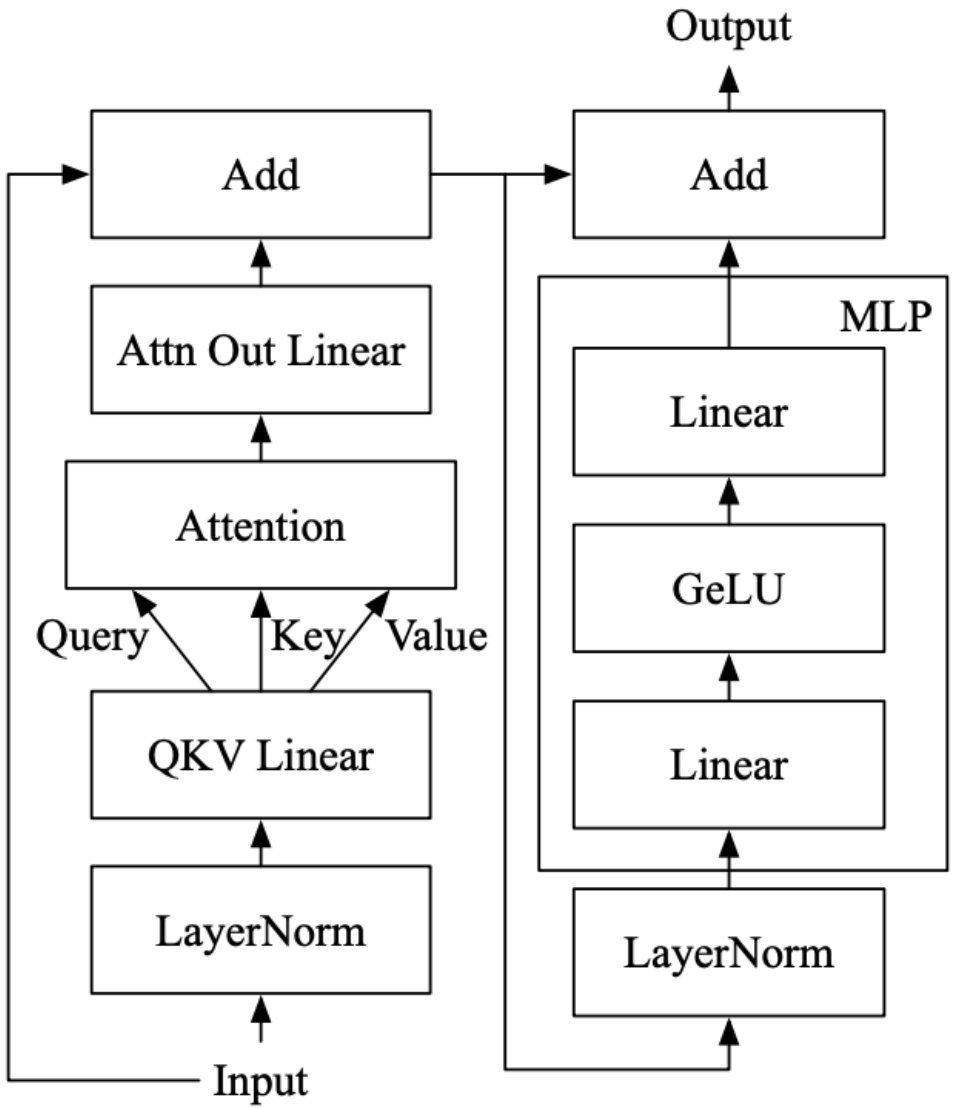
- Bin-packing problem: pack model sessions (model, SLO) to GPUs
- Optimization goal: minimize total number of GPUs
- Constraint: requests need to be served within latency SLOs
- More complex than bin packing due to
  - Change the batch size (squishy tasks)
  - Need to meet latency SLO

Now everything has changed in LLM

- One GPU holds many models -> One model occupies many GPUs
- Request is the unit of inference operation -> Token is the unit of inference operation



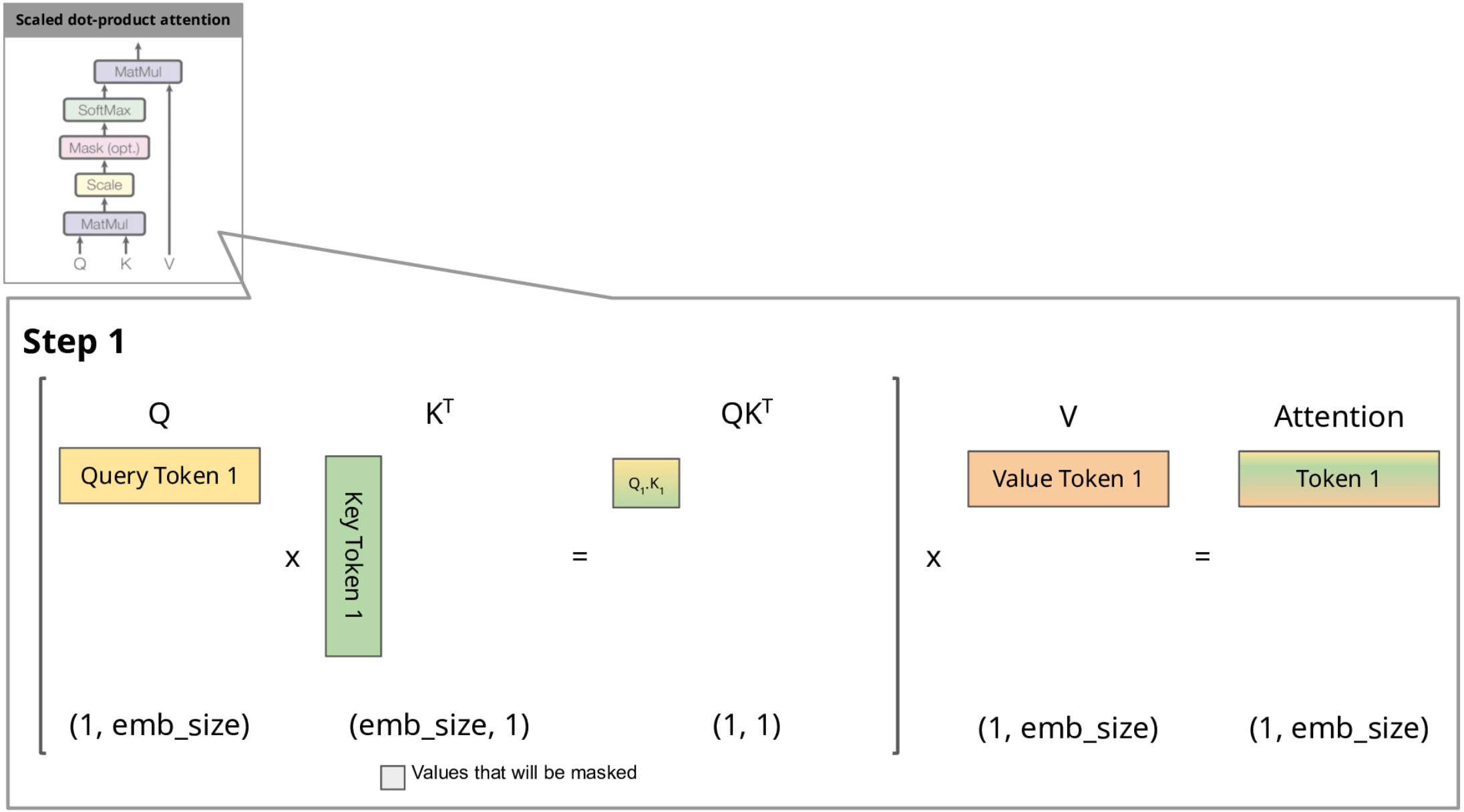
(a) A computation graph representing an inference procedure using a GPT model. The graph does not depict layers other than Transformer layers (e.g., embedding) for simplicity.



(b) A Transformer layer used in GPT.

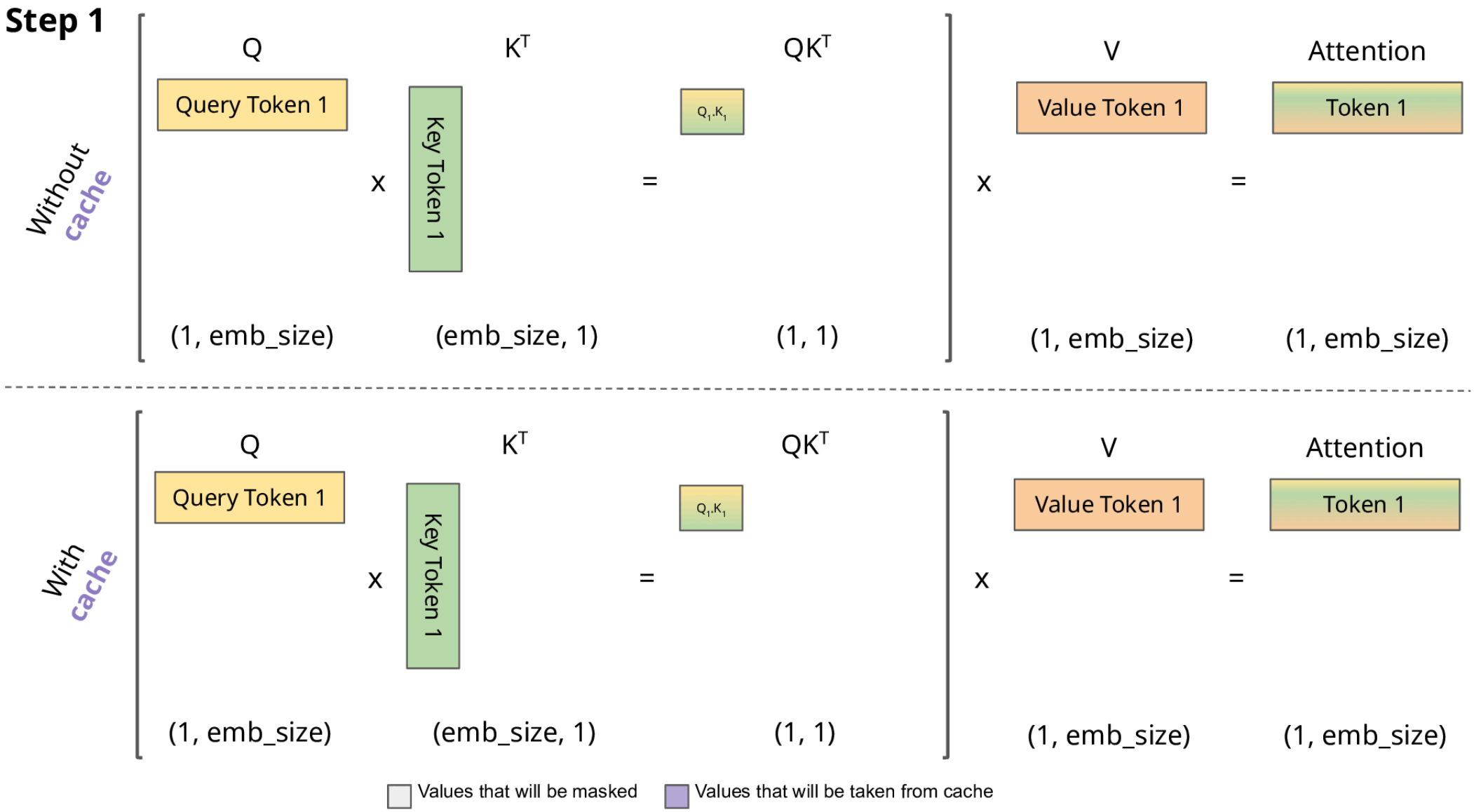
Inference

- Prefilling
  - Given a context, generate the first token
  - Save keys and values
  - This step is efficient when context is long, because we can batch in the sequence dimension. Similar to forward computing in training
- Decoding
  - Generate one token at a time
  - Reuse most of the key and values computed in the previous iteration
  - Save keys and values
  - This step is memory IO bound. We need to batch across requests!

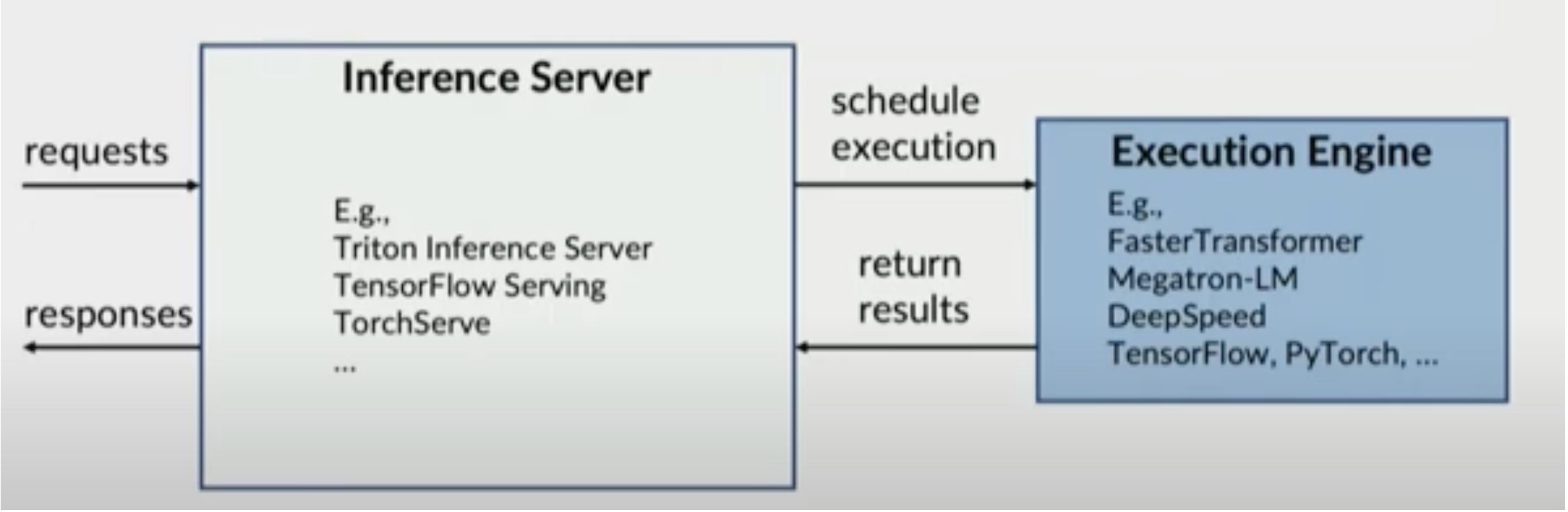


Zoom-in! (simplified without Scale and Softmax)

From “transformers kv caching explained.



From “transformers kv caching explained.


Inference Server
requests
responses
E.g.,
Triton Inference Server
TensorFlow Serving
TorchServe
...
schedule
execution
return
results
Execution Engine
E.g.,
FasterTransformer
Megatron-LM
DeepSpeed
TensorFlow, PyTorch, ...

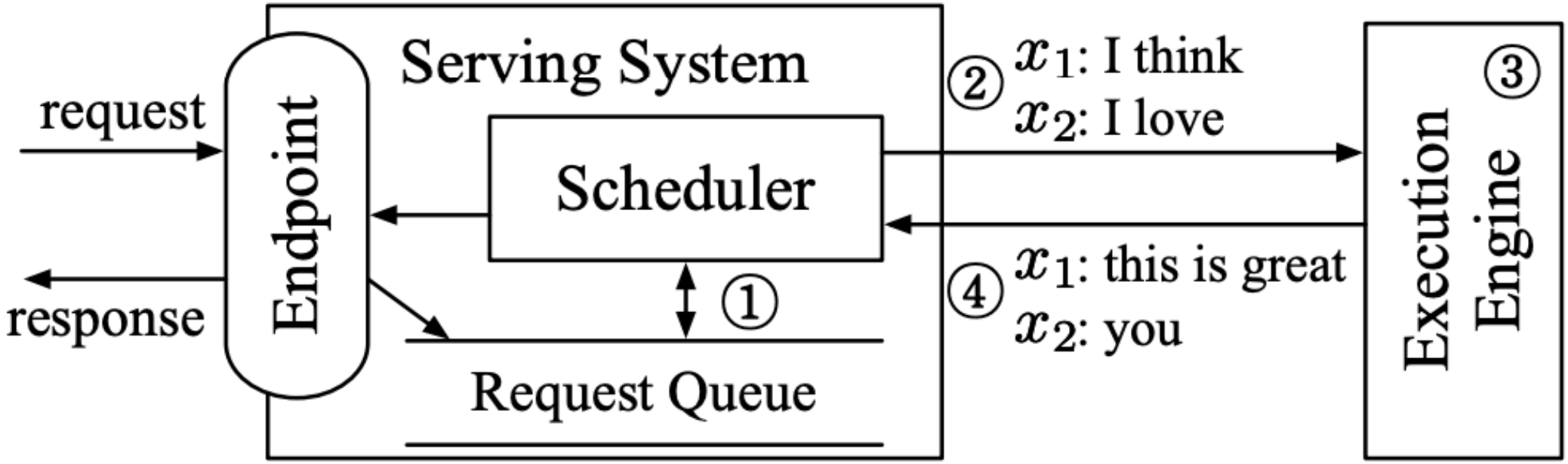


Figure 2: Overall workflow of serving a generative language model with existing systems.

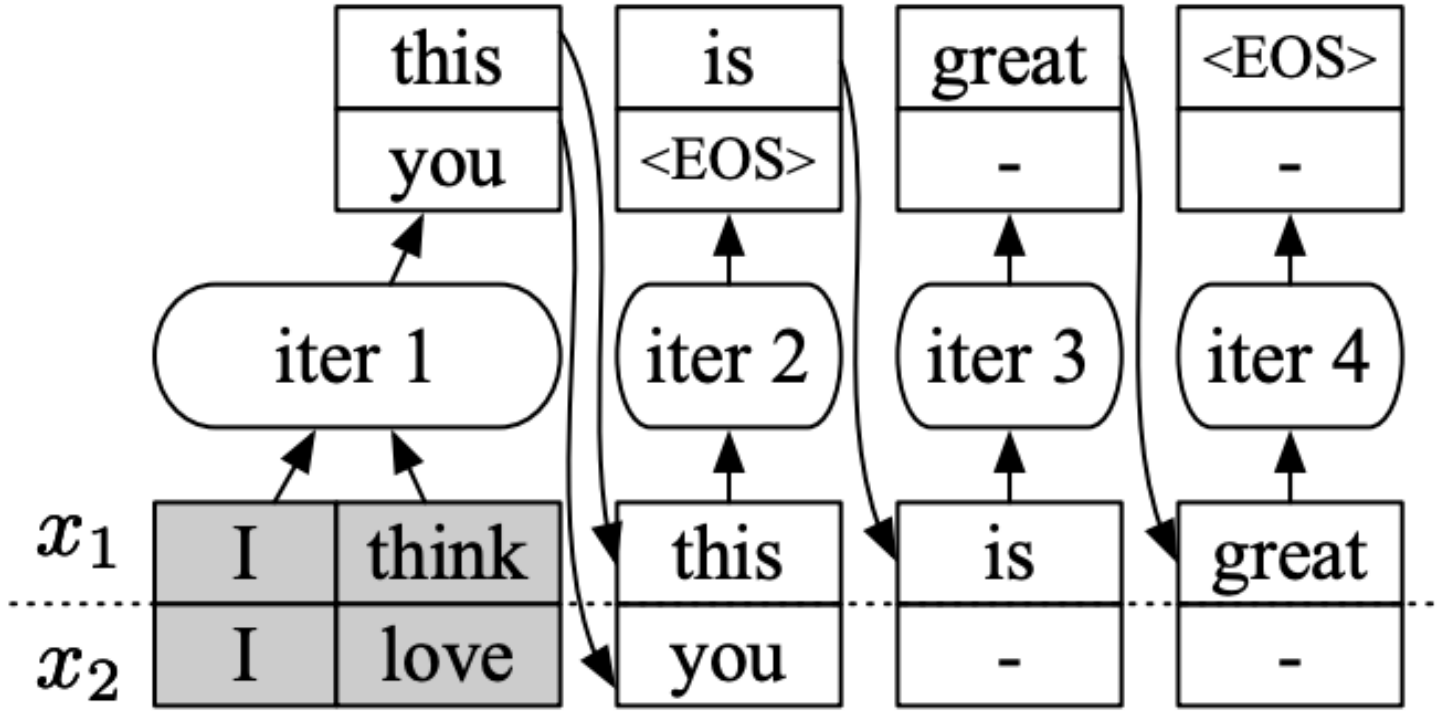
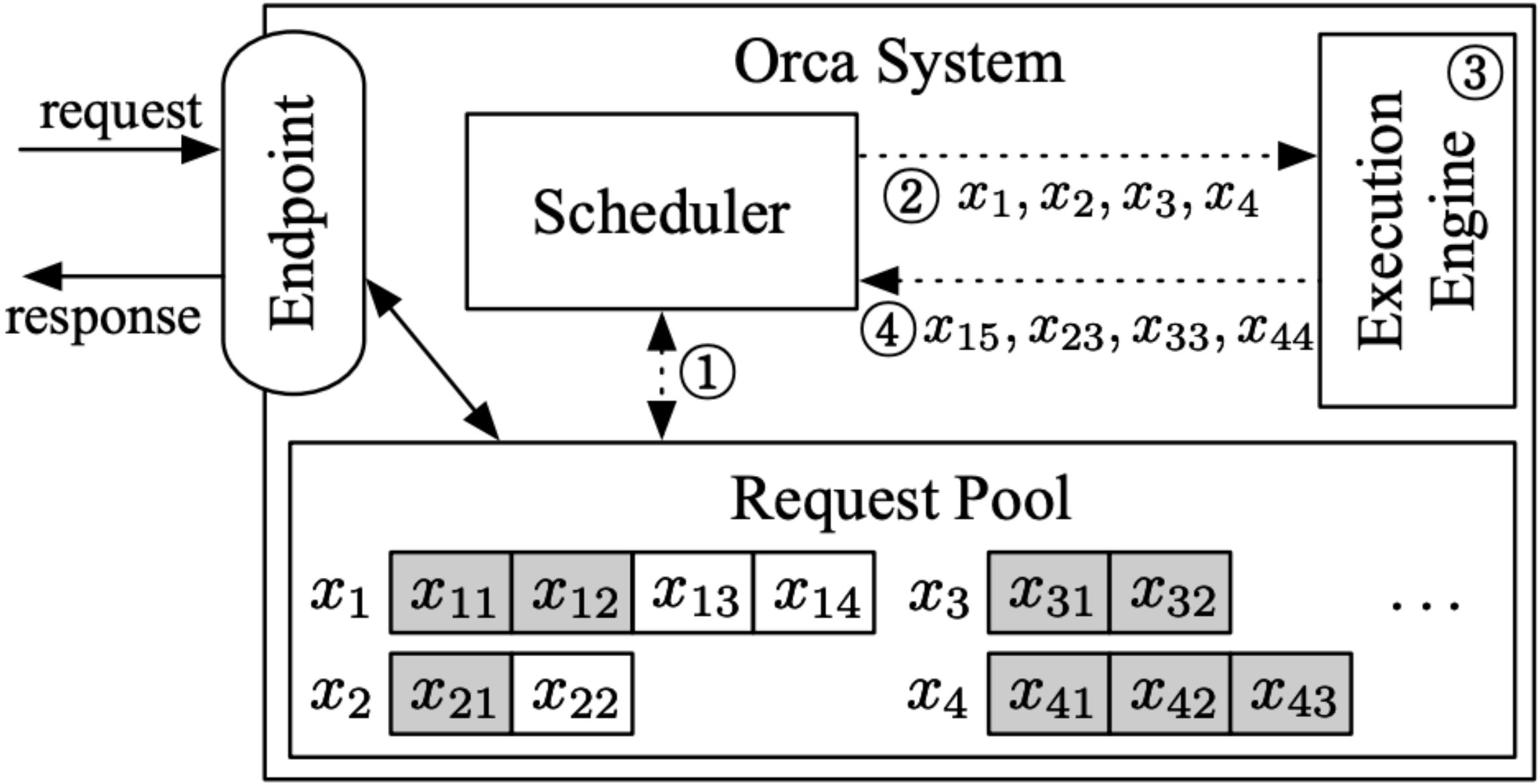


Figure 3: An illustration for a case where the requests have the same input length but some requests finish earlier than others. Shaded tokens represent input tokens. "-" denotes inputs and outputs of extra computation imposed by the scheduling.


Orca System
request
response
Endpoint
Scheduler
② x_1, x_2, x_3, x_4
④ $x_{15}, x_{23}, x_{33}, x_{44}$
Execution Engine ③
①
Request Pool
x_1 x_{11} x_{12} x_{13} x_{14}
x_2 x_{21} x_{22}
x_3 x_{31} x_{32}
x_4 x_{41} x_{42} x_{43}
...

Iteration-level scheduling

1. Selects requests to run next
2. Invokes the engine to execute one iteration for the selected requests
3. Receives execution results for the scheduled iteration

Newly arrived request get a chance to start processing in the next iteration.

Batching an arbitrary set of requests

- Challenges
  1. Both requests are prefill, but their lengths are different
  2. Both requests are decode, but they process different index
  3. One request is prefill, one request is decode

Batching requires identical operations to a “batch” of input

Selective batching

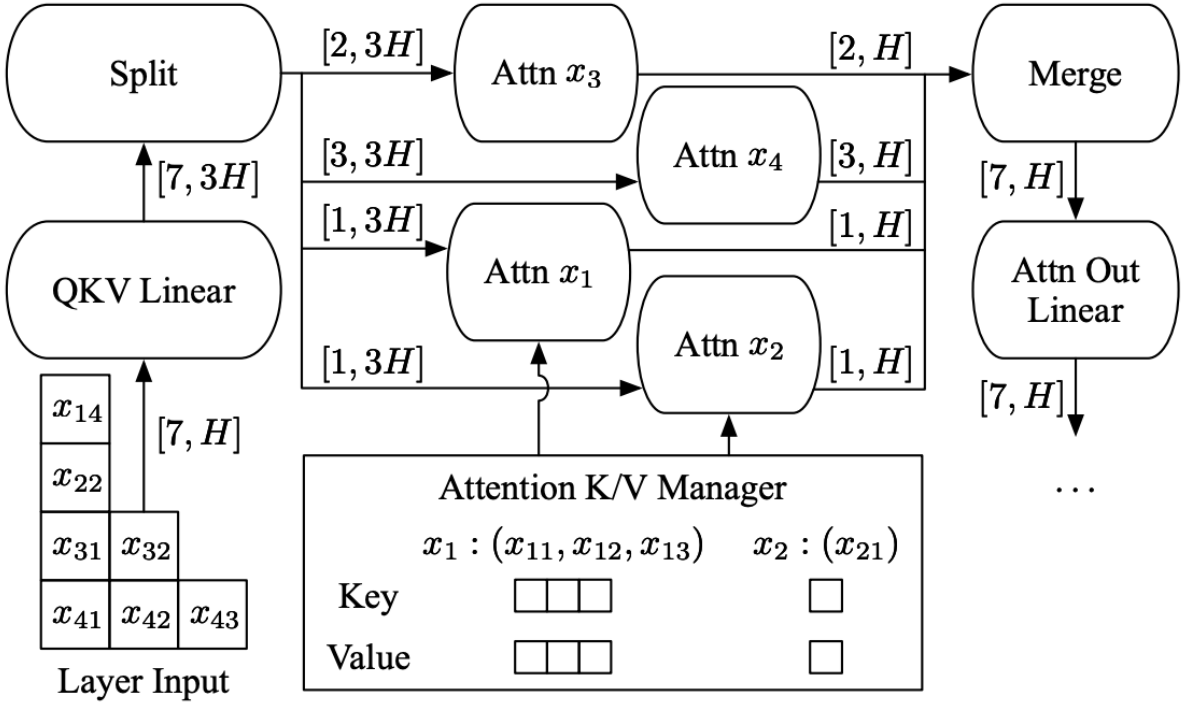


Figure 5: An illustration of ORCA execution engine running a Transformer layer on a batch of requests with selective batching. We only depict the QKV Linear, Attention, and Attention Out Linear operations for simplicity.

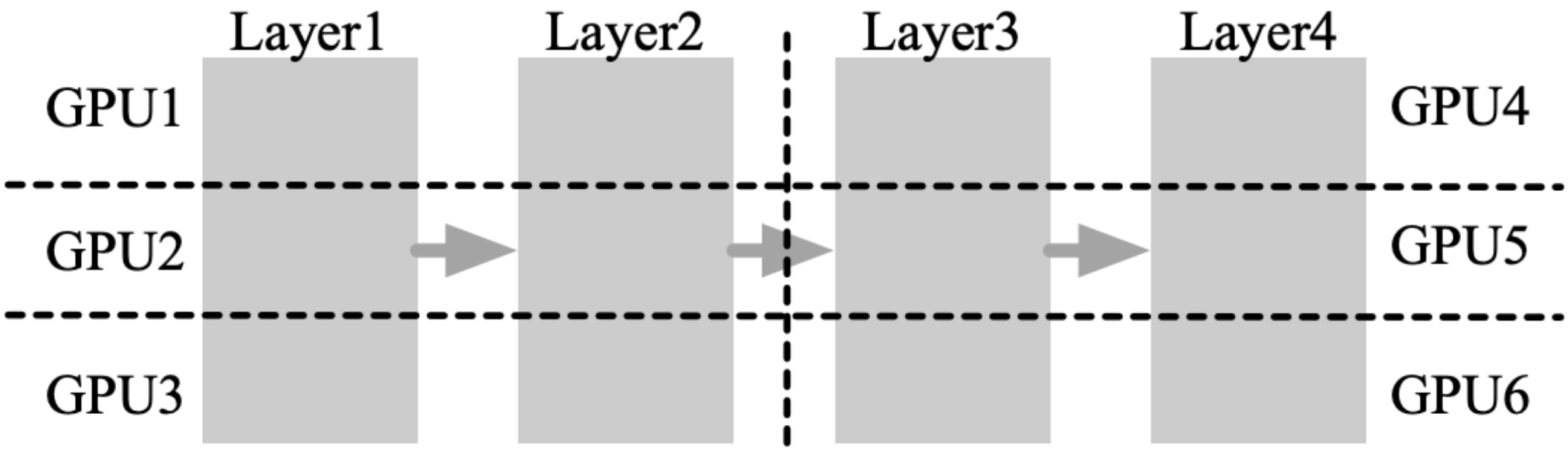


Figure 6: An example of intra- and inter- layer parallelism. A vertical dotted line indicates partitioning between layers and a horizontal line indicates partitioning within a layer.

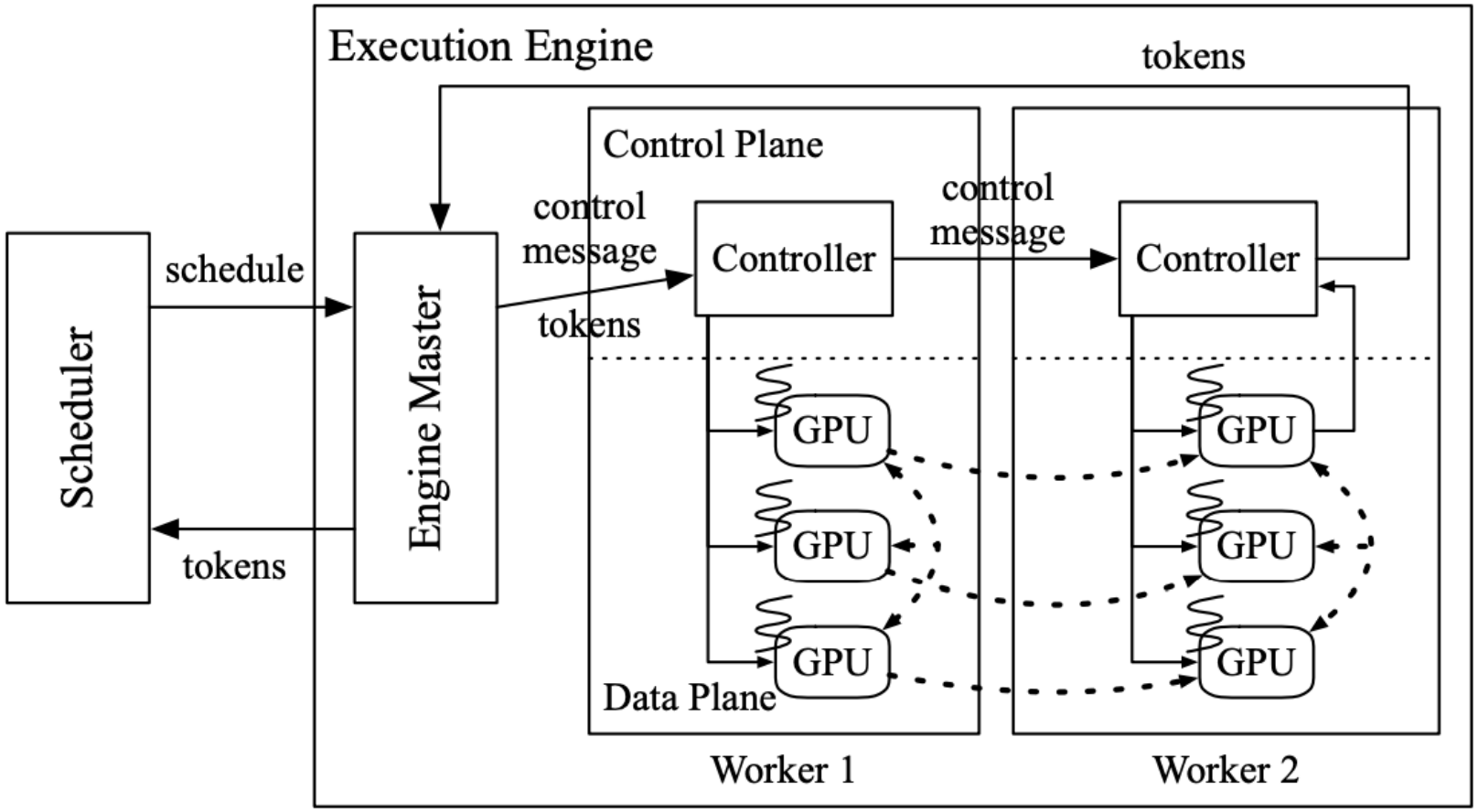


Figure 7: An illustration of the distributed architecture of ORCA's execution engine using the parallelization configuration shown in Figure 6. For example, the first inter-layer partition (Layer1 and Layer2) in Figure 6 is assigned to Worker1, while the second partition is assigned to Worker2.

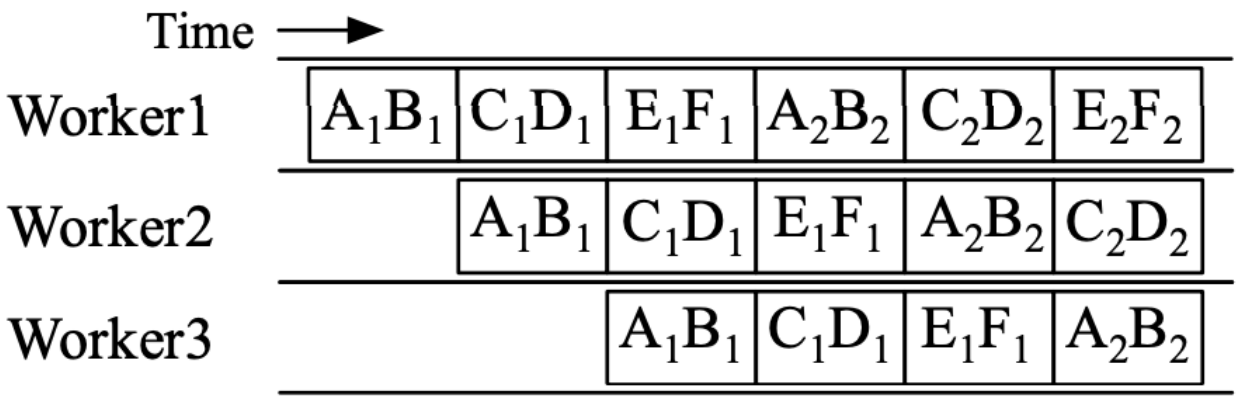
Scheduling policy

We define the iteration-level FCFS property for workloads with multi-iteration characteristics as follows: for any pair of requests (x_i, x_j) in the request pool, if x_i has arrived earlier than x_j, x_i should have run the same or more iterations than x_j. Note that some late-arrived requests may return earlier to clients if the late request requires a smaller number of iterations to finish.

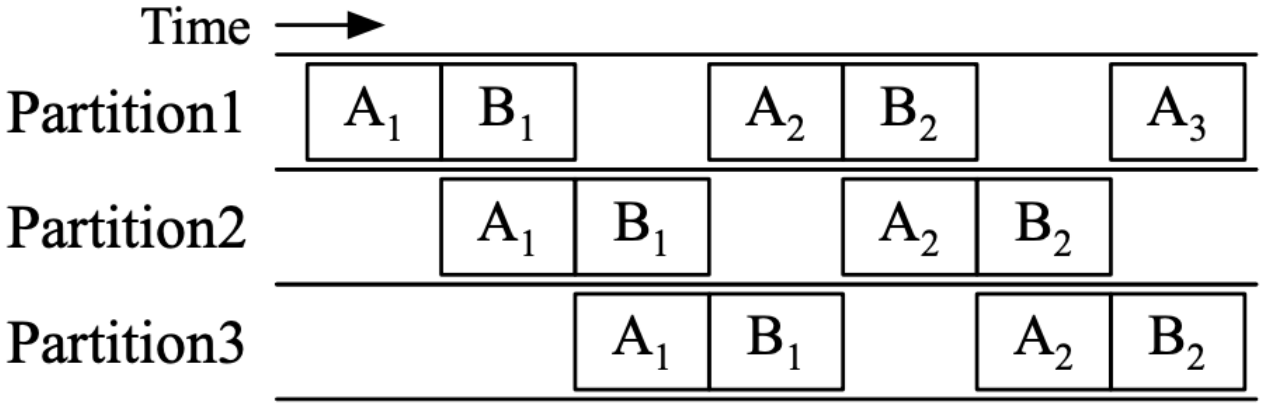
Algorithm 1: ORCA scheduling algorithm

```
Params: n_workers: number of workers, max_bs:
        max batch size, n_slots: number of K/V slots
n_scheduled ← 0
n_rsrv ← 0
while true do
    batch, n_rsrv ← Select(request_pool, n_rsrv)
    schedule engine to run one iteration of
    the model for the batch
    foreach req in batch do
        req.state ← RUNNING
    n_scheduled ← n_scheduled + 1
    if n_scheduled = n_workers then
        wait for return of a scheduled batch
        foreach req in the returned batch do
            req.state ← INCREMENT
            if finished(req) then
                n_rsrv ← n_rsrv − req.max_tokens
        n_scheduled ← n_scheduled − 1

def Select(pool, n_rsrv):
    batch ← {}
    pool ← {req ∈ pool | req.state ≠ RUNNING}
    SortByArrivalTime(pool)
    foreach req in pool do
        if batch.size() = max_bs then break
        if req.state = INITIATION then
            new_n_rsrv ← n_rsrv + req.max_tokens
            if new_n_rsrv > n_slots then break
            n_rsrv ← new_n_rsrv
        batch ← batch ∪ {req}
    return batch, n_rsrv
```



(a) ORCA execution pipeline.



(b) FasterTransformer execution pipeline.

Figure 8: Comparison of the use of pipeline parallelism in ORCA and FasterTransformer where X_i is the i-th iteration of request X.